

Retrieval-Augmented Generation (RAG)

Dr. Sisay Chala

September 23, 2026

Learning Objectives

By the end of this presentation, you will understand:

- What Retrieval-Augmented Generation (RAG) is

Learning Objectives

By the end of this presentation, you will understand:

- What Retrieval-Augmented Generation (RAG) is
- Why RAG is useful

Learning Objectives

By the end of this presentation, you will understand:

- What Retrieval-Augmented Generation (RAG) is
- Why RAG is useful
- What document chunking means

Learning Objectives

By the end of this presentation, you will understand:

- What Retrieval-Augmented Generation (RAG) is
- Why RAG is useful
- What document chunking means
- What embeddings are

Learning Objectives

By the end of this presentation, you will understand:

- What Retrieval-Augmented Generation (RAG) is
- Why RAG is useful
- What document chunking means
- What embeddings are
- What FAISS does

Learning Objectives

By the end of this presentation, you will understand:

- What Retrieval-Augmented Generation (RAG) is
- Why RAG is useful
- What document chunking means
- What embeddings are
- What FAISS does
- How retrieval works

Learning Objectives

By the end of this presentation, you will understand:

- What Retrieval-Augmented Generation (RAG) is
- Why RAG is useful
- What document chunking means
- What embeddings are
- What FAISS does
- How retrieval works
- How retrieved information is given to an LLM

Learning Objectives

By the end of this presentation, you will understand:

- What Retrieval-Augmented Generation (RAG) is
- Why RAG is useful
- What document chunking means
- What embeddings are
- What FAISS does
- How retrieval works
- How retrieved information is given to an LLM
- How to build a simple RAG pipeline in Python

What is RAG?

RAG = Retrieval-Augmented Generation

Instead of asking an LLM to answer only from its training knowledge:

Question $\rightarrow$ LLM $\rightarrow$ Answer

RAG first retrieves relevant information from external documents:

Question $\rightarrow$ Retrieve relevant information $\rightarrow$ LLM $\rightarrow$ Answer

Why Use RAG?

Imagine we have a company handbook:

Company Policy

Employees receive 30 days of paid vacation per year.

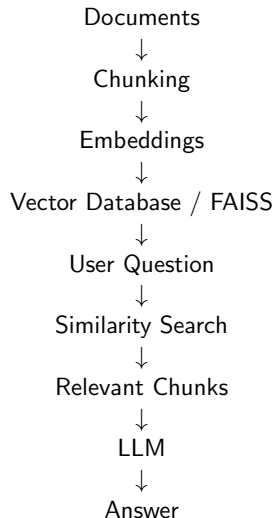
A user asks:

How many vacation days do employees receive?

A general-purpose LLM may not know this private company policy.

RAG retrieves the relevant passage and provides it to the LLM.

Basic RAG Architecture



Step 1: Load Documents

Create a simple document:

Listing 1: documents.txt

```
RAG stands for Retrieval-Augmented Generation.
```

```
RAG systems retrieve relevant information from a  
knowledge base and provide that information to a  
language model as context.
```

```
The main stages of a RAG system are document loading,  
chunking, embedding, retrieval, and generation.
```

Load it with Python:

```
with open("documents.txt", "r",  
          encoding="utf-8") as f:  
    text = f.read()
```

Step 2: Chunking

Large documents are divided into smaller pieces called **chunks**.

```
def chunk_text(text, chunk_size=300):  
    words = text.split()  
  
    chunks = []  
  
    for i in range(0, len(words), chunk_size):  
        chunk = " ".join(  
            words[i:i + chunk_size]  
        )  
        chunks.append(chunk)  
  
    return chunks
```

What is chunk_size?

In our simple implementation:

```
def chunk_text(text, chunk_size=300):
```

chunk_size=300 means:

up to 300 words per chunk

For example, with a 750-word document:

- Chunk 1: words 1–300
- Chunk 2: words 301–600
- Chunk 3: words 601–750

Why Does Chunk Size Matter?

Chunks that are too small

- May lose important context
- Retrieved information may be incomplete

Chunks that are too large

- May contain irrelevant information
- Retrieval becomes less precise
- More context is sent to the LLM

In real systems, chunks often use:

- Paragraphs
- Sections
- Sentences
- Token-based limits
- Overlapping chunks

Step 3: Embeddings

An **embedding** converts text into a numerical vector.

For example:

"RAG retrieves documents"

becomes something like:

$[0.12, -0.44, 0.81, 0.03, \dots]$

The vector represents semantic information about the text.

Similar pieces of text tend to have similar vectors.

Creating Embeddings with Python

Install Sentence Transformers:

```
pip install sentence-transformers
```

Load an embedding model:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer(
    "all-MiniLM-L6-v2"
)
```

Create embeddings:

```
embeddings = model.encode(chunks)

print(embeddings.shape)
```

For example:

(2, 384)

means 2 chunks with 384-dimensional vectors.

Step 4: Indexing using FAISS

FAISS = Facebook AI Similarity Search

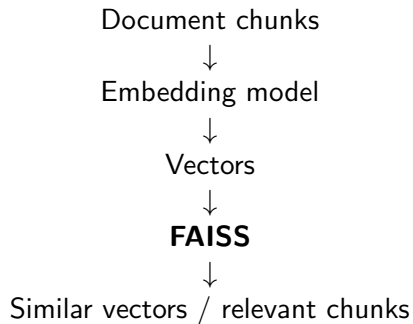
FAISS is a library developed by Meta for **efficient similarity search over vectors**.

In RAG, FAISS can store document embeddings and quickly find vectors similar to a user's question.

Think of FAISS as:

a search engine for embeddings

FAISS in the RAG Pipeline



FAISS does **not** generate answers.
It performs similarity search.

Creating a FAISS Index

Install FAISS:

```
pip install faiss-cpu
```

Create an index:

```
import faiss

dimension = embeddings.shape[1]

index = faiss.IndexFlatL2(dimension)
```

Add document embeddings:

```
index.add(embeddings)
```

What Does IndexFlatL2 Mean?

Our example uses:

```
faiss.IndexFlatL2(dimension)
```

L2 refers to Euclidean distance.

Conceptually:

$$d(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$$

Smaller distance means the vectors are closer.

FAISS provides many other index types optimized for different datasets and performance requirements.

Step 5: Retrieval

Suppose the user asks:

What are the main stages of RAG?

First, embed the question:

```
question_embedding = model.encode(  
    [question]  
)
```

Then search FAISS:

```
distances, indices = index.search(  
    question_embedding,  
    k=2  
)
```

k=2 means:

retrieve the 2 most similar chunks

Retrieving the Actual Text

FAISS returns the indices of the matching vectors.
We use those indices to retrieve the original chunks:

```
retrieved_chunks = [  
    chunks[i]  
    for i in indices[0]  
]
```

Now we have the information that is most relevant to the question.

Step 6: Give Context to the LLM

Combine the retrieved chunks:

```
context = "\n\n".join(
    retrieved_chunks
)
```

Build a prompt:

```
prompt = f"""
Answer the question using only the context below.

Context:
{context}

Question:
{question}

If the answer cannot be found in the context, say:
"I don't know based on the provided documents."
"""
```

Step 7: Generate the Answer

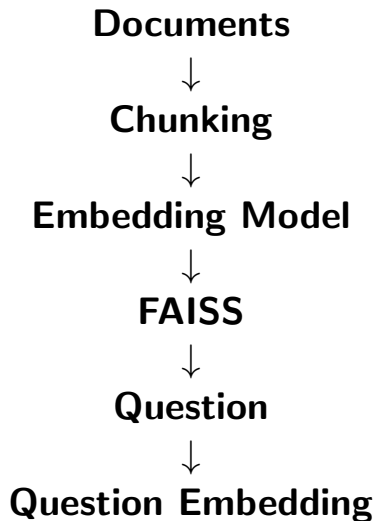
Use an LLM:

```
from openai import OpenAI

client = OpenAI()

response = client.responses.create(
    model="gpt-5.6",
    input=prompt
)

print(response.output_text)
```



The Three Main Components

A simple RAG system combines three important components.

1. Embedding Model

Converts text into vectors.

2. Vector Search

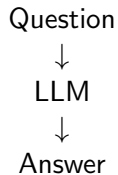
Finds document vectors similar to the question. FAISS can perform this task.

3. LLM

Uses the retrieved context to generate a natural-language answer.

Traditional LLM vs RAG

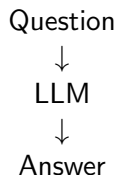
Traditional LLM



The model relies primarily on its existing knowledge.

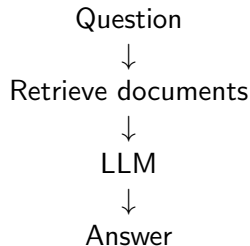
Traditional LLM vs RAG

Traditional LLM



The model relies primarily on its existing knowledge.

RAG



The model receives relevant external context.

Where Can RAG Be Used?

RAG is useful when an LLM needs access to external or private information.

- Company knowledge bases
- Internal documentation
- Customer-support systems
- Research assistants
- Product manuals
- Technical documentation
- Question answering over PDFs
- Internal wikis

Limitations of our RAG Example

A production RAG system needs to consider:

- Better document parsing
- Better chunking strategies
- **Chunk overlap**
- **Metadata**
- Better vector indexes
- Hybrid search
- Reranking
- Retrieval evaluation
- Answer evaluation
- Source citations

Chunk Overlap

Instead of completely independent chunks:

Chunk 1: 1–300

Chunk 2: 301–600

we can use overlap:

Chunk 1: 1–300

Chunk 2: 250–550

Chunk 3: 500–800

Overlap helps preserve information that occurs near chunk boundaries.

Metadata

Instead of storing only the text:

```
chunk
```

we can store additional information:

```
{  
  "text": "...",  
  "document": "handbook.pdf",  
  "page": 42,  
  "section": "Vacation Policy"  
}
```

This makes it possible to provide source information with the generated answer.

Example Implementation

You can find the example implementation on GitHub:

<https://github.com/AI-Cloud-AWS/rag-example>

In order to execute this example, you need:



- OpenAI key

RAG is not a special type of LLM.

It is a pipeline combining

Information Retrieval

+

Language Generation

Questions?